디지털 논리

Number Systems

이 상 화

Positional Number System

- **Positional number system (진법, 자릿수 방식)**
  - 자릿수에 따라 base(기저 숫자)의 거듭제곱으로 표현하여 숫자를 표기
- **Ex: 십진수 (decimal)**
  - base=10
  - Ex: $543 = 5*10^2 + 4*10^1 + 3*10^0$
- **Radix(base) *r***

$$d_n \dots d_2 d_1 d_{0_{(r)}} = \sum_{k=0}^{n} d_k r^{k-1},$$

$$d_k = 0,1,2, \dots r-1$$

Positional Number System

- **Examples with fraction**

Decimal (십진수)

$$953.78_{10} = 9\times10^2 + 5\times10^1 + 3\times10^0 + 7\times10^{-1} + 8\times10^{-2}$$

Binary (이진수)

$$1011.11_2 = 1\times2^3 + 0\times2^2 + 1\times2^1 + 1\times2^0 + 1\times2^{-1} + 1\times2^{-2}$$

$$= 8 + 0 + 2 + 1 + \frac{1}{2} + \frac{1}{4} = 11\frac{3}{4} = 11.75_{10}$$

Hexadecimal (16진수)

$$A2F_{16} = 10\times16^2 + 2\times16^1 + 15\times16^0 = 2560 + 32 + 15 = 2607_{10}$$

Radix(Base) R:

$$N = (a_4 a_3 a_2 a_1 a_0 . a_{-1} a_{-2} a_{-3})_R$$

$$= a_4\times R^4 + a_3\times R^3 + a_2\times R^2 + a_1\times R^1 + a_0\times R^0$$

$$+ a_{-1}\times R^{-1} + a_{-2}\times R^{-2} + a_{-3}\times R^{-3}$$

=> Conversion Radix R to decimal number

Decimal to Radix r number (1)

- **Nested expansion**
  - Quotient – remainder (몫-나머지)
  - 몫이 0이 될 때까지 반복하여 R로 나누면서 나오는 나머지를 하위 digit에 순서대로 적용

$$N = (a_n a_{n-1} \cdots a_2 a_1 a_0)_R = a_n R^n + a_{n-1} R^{n-1} + \cdots + a_2 R^2 + a_1 R^1 + a_0$$

$$\frac{N}{R} = a_n R^{n-1} + a_{n-1} R^{n-2} + \cdots + a_2 R^1 + a_1 = Q_1, \text{remainder } a_0$$

$$\frac{Q_1}{R} = a_n R^{n-2} + a_{n-1} R^{n-3} + \cdots + a_3 R^1 + a_2 = Q_2, \text{remainder } a_1$$

$$\frac{Q_2}{R} = a_n R^{n-3} + a_{n-1} R^{n-4} + \cdots + a_3 = Q_3, \text{remainder } a_2$$

=> Repeat until Q is zero

Decimal to Radix r number (2)

Ex: Decimal (53) to Binary (r=2) Conversion

2	53	
2	26	rem. = 1 = a_0
2	13	rem. = 0 = a_1
2	6	rem. = 1 = a_2
2	3	rem. = 0 = a_3
2	1	rem. = 1 = a_4
	0	rem. = 1 = a_5

$$53_{10} = 110101_2$$

Conversion of Decimal Fraction (1)

- Conversion of a decimal fraction to Base-R

= > R을 곱하면서 정수로 나오는 값을 순서대로 digit 적용

$$F = (.a_{-1}a_{-2}a_{-3}\cdots a_{-m})_R = a_{-1}R^{-1} + a_{-2}R^{-2} + a_{-3}R^{-3} + \cdots + a_{-m}R^{-m}$$

$$FR = a_{-1} + a_{-2}R^{-1} + a_{-3}R^{-2} + \cdots + a_{-m}R^{-m+1} = a_{-1} + F_1$$

$$F_1R = a_{-2} + a_{-3}R^{-1} + \cdots + a_{-m}R^{-m+2} = a_{-2} + F_2$$

$$F_2R = a_{-3} + \cdots + a_{-m}R^{-m+3} = a_{-3} + F_3$$

=> 소수 부분이 없어질 때까지 반복

Example: Binary (base=2) fraction

$$\begin{array}{rr} F = & .625 \\ & \underline{\times \quad 2} \\ & 1.250 \\ & (a_{-1} = 1) \end{array} \qquad \begin{array}{rr} F_1 = & .250 \\ & \underline{\times \quad 2} \\ & 0.500 \\ & (a_{-2} = 0) \end{array} \qquad \begin{array}{rr} F_2 = & .500 \\ & \underline{\times \quad 2} \\ & 1.000 \\ & (a_{-3} = 1) \end{array}$$

$$.625_{10} = .101_2$$

Conversion of Decimal Fraction (2)

Example: infinite fraction

$$
\begin{array}{r}
.7 \\
\underline{\quad 2} \\
(1).4 \\
\underline{\quad 2} \\
(0).8 \\
\underline{\quad 2} \\
(1).6 \\
\underline{\quad 2} \\
(1).2 \\
\underline{\quad 2} \\
(0).4 \\
\underline{\quad 2} \\
(0).8
\end{array}
$$

.4 was previously obtained => infinitely repeated

$$0.7_{10} \rightarrow 0.1\underline{0110}\,\underline{0110}\,\underline{0110}\cdots_2$$

Conversion – General Case

- **First convert to base-10, and convert to base-r**

Ex: Convert $231.3_{(4)}$ to base−7

$$231.3_4 = 2\times16+3\times4+1+\frac{3}{4} = 45.75_{10}$$

$$\begin{array}{rl} 7\,\big/\,45 & \\ \hline 7\,\big/\,6 & \text{rem.}3 \\ \hline 0 & \text{rem.}6 \end{array}$$

$$\begin{array}{r} .75 \\ 7 \\ \hline (5).25 \\ 7 \\ \hline (1).75 \\ 7 \\ \hline (5).25 \\ 7 \\ \hline (1).75 \end{array}$$

$$45.75_{10} = 63.5151\cdots_7$$

디지털 논리

Binary Representation and Addition

이 상 화

Binary Arithmetic (1)

Binary addition

$$0+0=0$$

$$1+0=1$$

$$0+1=1$$

$$1+1=0$$

$$\begin{array}{r} 1101 \\ +1011 \\ \hline 11000 \end{array}$$

Binary subtraction

$$0-0=0$$

$$1-0=1$$

$$0-1=1$$

$$1-1=0$$

$$\begin{array}{r} 11101 \\ -\ \ 1011 \\ \hline 10010 \end{array}$$

Binary Arithmetic (2)

Binary multiplication

$$1 \cdot 1 = 1$$
$$1 \cdot 0 = 0$$
$$0 \cdot 0 = 0$$

$$\begin{array}{r} 101 \\ \times\ 10 \\ \hline 000 \\ 101 \\ \hline 1010 \end{array}$$

Binary division

$$1/1 = 1$$
$$0/1 = 0$$

$$\begin{array}{r|l} & 10100 \\ \hline 101 & 1100101 \\ & 101 \\ & 101 \\ & 101 \\ \hline & 001 \end{array}$$

Negative Numbers (1)

- **Signed-magnitude representation**
  - MSB (leftmost): sign bit
  - The other bits: absolute value
- Examples

  $\underline{0}1010101_2 = +85_{10}$

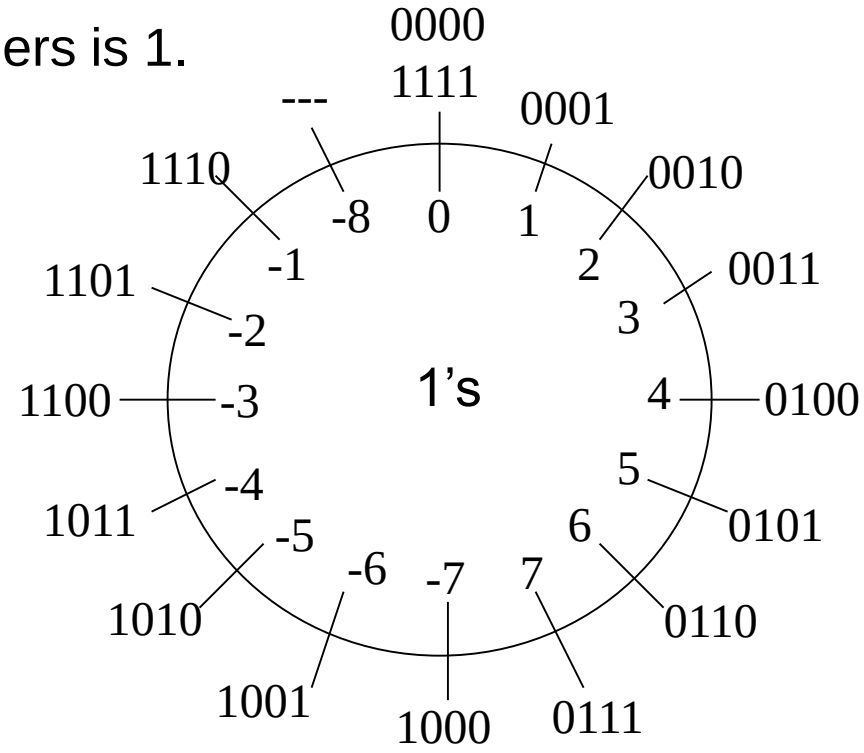
  $\underline{1}1010101_2 = -85_{10}$
- Range: $[-2^{n-1}+1,\ 2^{n-1}-1]$:
  - For n=4 bits : -7 ~ +7
- Limitation
  - Two zeros: 0000 (+0), 1000(-0)
  - $\pm 2^{n-1}$ are not defined.
    - For 4 bits : 8, -8 are not defined.

Negative Numbers (2)

- 1's complement representation

$$-N = \overline{N} = (2^n - 1) - N$$

- Equivalent to complement bit by bit
  - Ex: -5: 0101 → 1010
  - Sign bit of all negative numbers is 1.
- Range: $[-2^{n-1}+1,\ 2^{n-1}-1]$
  - n=4: -7 ~ +7
- Limitation
  - Two zeros
    - n=4: 0000(+0), 1111(-0)
  - $\pm 2^{n-1}$ are not defined
    - n=4: -8 is not defined

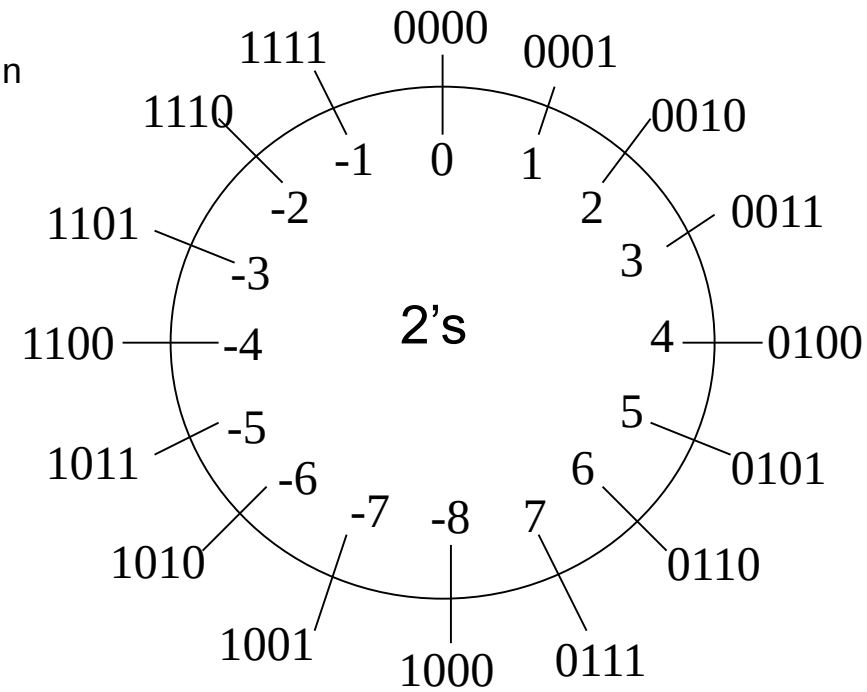


Negative Numbers (2)

- 2's complement representation

$$-N = N^* = 2^n - N$$

- Sign bit of all negative numbers is 1.
  - 0 is included in the positive number
- Range: $[-2^{n-1}, 2^{n-1}-1]$
  - Completely represented in 2^n
  - n=4: -8 ~ +7



Negative Numbers (4)

- We use 2's complement representation for negative numbers
- 2's complement representation

  $N^* = 2^n - N$

- 1's complement representation

  $\bar{N} = (2^n - 1) - N$

- By combining with two complement representations

  $N^* = \bar{N} + 1$

  - First 1's complement (bit by bit complement), then add binary 1
  - Ex: N(88)=0101100
    - -N(-88): 1010011→+1→1010100

2's Complement Addition

Positive number 인 것처럼 연산하고,
sign bit의 carry는 무시 (단, overflow 검사 필요)

+3	0011	−2	1110
+ +4	+0100	+ −6	+1010
+7	0111	−8	1000
+6	0110	+4	0100
+ −3	+1101	+ −7	+1001
+3	0011	−3	1101

- overflow

−6	1010
+ −3	+1101
−9	0111 = +7

How to check the overflow??

2's Complement Subtraction

- 2's complement negative number로 변환하고 더함.
- C_{in}: 1's complement + 1 => 2's complement

		1	(C_{in})
+6	0110	0110	
− +3	−0011	1100	
+3		0011	

Sign bit carry 무시

		1	(C_{in})
+6	0110	0110	
− −1	−1111	0000	
+7		0111	

1's Complement Addition

Positive number 인 것처럼 연산하고,
sign bit의 carry (end-around carry) 는 rightmost bit에 더함.
(단, overflow 검사 필요)

+3	0011
+ +4	+0100
+7	0111

−2	1101
+ −5	+1010
−7	(1) 0111
	1
	1000

+6	0110
+ −3	+1100
+3	(1) 0010
	1
	0011

+4	0100
+ −7	+1000
−3	1100

Binary Codes (1)

- Each digit is represented by a binary number
  - No need of operation, fixed length
- Weighted codes
  - 4bits code: $N = w_3 a_3 + w_2 a_2 + w_1 a_1 + w_0 a_0$
  - 8-4-2-1 (BCD), 6-3-1-1 codes
- Excess-3 code
  - Add 3 (0011) to BCD code
- 2 out of 5 code
  - 2 out of 5 bits are 1.
  - Useful for error check
- Gray codes
  - Codes for successive decimal digits differ in exactly one bit

Binary Codes (2)

Each digit is represented by binary number.

9 3 7.2 5

1001 0011 0111 . 0010 0101

Decimal Digit	8-4-2-1 Code (BCD)	6-3-1-1 Code	Excees-3 Code	2-out-of-5 Code	Gray Code
0	0000	0000	0011	00011	0000
1	0001	0001	0100	00101	0001
2	0010	0011	0101	00110	0011
3	0011	0100	0110	01001	0010
4	0100	0101	0111	01010	0110
5	0101	0111	1000	01100	1110
6	0110	1000	1001	10001	1010
7	0111	1001	1010	10010	1011
8	1000	1011	1011	10100	1001
9	1001	1100	1100	11000	1000

Weighted codes

Alphanumeric Codes

- **ASCII code**
  - American Standard Code for Information Interchange
  - 7 bit, 128 characters
- **Parity bit to detect errors**
  - Use the MSB (leftmost bit)
  - Even parity ASCII A 1000001 => 01000001
  - Odd parity => 11000001